```
    size_type size() const;

    T& front();
    const T& front() const;
    T& back();
    const T& back() const;

    void push(const T& x);
    void push(T&& x);
    void pop();
    template <class... Args> void emplace(Args&&... args);
};
```

If you ignore the construction, assignment and swap operations, you're left with three groups of operations: those that query the state of the whole queue (`empty()` and `size()`), those that query the elements of the queue (`front()` and `back()`), and those that modify the queue (`push()`, `pop()` and `emplace()`). This is the same as you had back in section 3.2.3 for the stack, and therefore you have the same issues regarding race conditions inherent in the interface. Consequently, you need to combine `front()` and `pop()` into a single function call, much as you combined `top()` and `pop()` for the stack. The code from listing 4.1 adds a new nuance, though: when using a queue to pass data between threads, the receiving thread often needs to wait for the data. Let's provide two variants on `pop()`: `try_pop()`, which tries to pop the value from the queue but always returns immediately (with an indication of failure) even if there wasn't a value to retrieve, and `wait_and_pop()`, which will wait until there's a value to retrieve. If you take your lead for the signatures from the stack example, your interface looks like the following.

Listing 4.3. The interface of your `threadsafe_queue`

```
#include <memory>                    <- For std::shared_ptr

template<typename T>
class threadsafe_queue
{
public:
    threadsafe_queue();
    threadsafe_queue(const threadsafe_queue&);
    threadsafe_queue& operator=(
        const threadsafe_queue&) = delete;      <- Disallow assignment for simplicity

    void push(T new_value);

    bool try_pop(T& value);                     <- (1)
    std::shared_ptr<T> try_pop();               <- (2)

    void wait_and_pop(T& value);
    std::shared_ptr<T> wait_and_pop();

    bool empty() const;
};
```

As you did for the stack, you've cut down on the constructors and eliminated assignment in order to simplify the code. You've also provided two versions of both `try_pop()` and `wait_for_pop()`, as before. The first overload of `try_pop()` (1) stores the retrieved value in the referenced variable, so it can use the return value for status; it returns `true` if it retrieved a value and `false` otherwise (see section A.2). The second overload (2) can't do this, because it returns the retrieved value directly. But the returned pointer can be set to `NULL` if there's no value to retrieve.